

Full-Stack Case Study: Drone Fleet Mission Control & Maintenance Tracker

Context

Company: SkyOps Ltd. -- a commercial drone services company providing aerial inspection services for wind turbines, solar farms, and power lines across Europe.

SkyOps operates a fleet of 150+ industrial drones. Each drone must undergo strict maintenance cycles (per aviation authority regulations), and every mission must be logged. Currently, operations are managed via spreadsheets and email -- leading to missed maintenance windows, untracked battery degradation, and compliance risks.

Your task is to build **SkyOps Mission Control** -- an internal tool for operations managers, drone pilots, and maintenance technicians.

Tech Stack

Layer	Technology
Backend	NestJS (TypeScript), TypeORM
Database	PostgreSQL
Frontend	React/Vue (TypeScript)

Testing	Jest (backend), Cypress or Playwright (frontend e2e)
API Style	REST

You may use any additional libraries you find necessary.

Requirements

1. Drone Registry

Manage the fleet inventory with full CRUD operations.

A drone has the following properties:

- Unique identifier
- Serial number (unique, must follow the format **SKY-XXXX-XXXX** where X is alphanumeric)
- Model (one of: PHANTOM_4, MATRICE_300, MAVIC_3_ENTERPRISE)
- Status (one of: AVAILABLE, IN_MISSION, MAINTENANCE, RETIRED)
- Total flight hours
- Last maintenance date
- Next maintenance due date
- Registration timestamp

Rules:

- Maintenance is required **every 50 flight hours OR every 90 days**, whichever comes first. The next maintenance due date should be calculated automatically.
- Only drones with AVAILABLE status can be assigned to missions.
- A drone with upcoming scheduled missions cannot be retired.

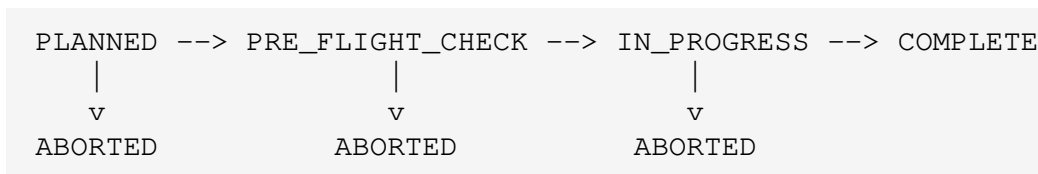
2. Mission Management

Schedule and track inspection missions.

A mission includes:

- Basic info (name, type, assigned drone, pilot name, site location)
- Mission type: WIND_TURBINE_INSPECTION, SOLAR_PANEL_SURVEY, or POWER_LINE_PATROL
- Scheduling info (planned start/end, actual start/end)
- Status tracking
- Flight hours logged upon completion
- Abort reason (if applicable)

Mission Lifecycle:



Rules:

- Missions must follow the valid state transitions shown above.
- When a mission starts (IN_PROGRESS), the drone status should reflect that it is in use.
- When a mission is completed, flight hours must be logged and added to the drone's total. The system should check if maintenance is now due.
- Aborting a mission requires a reason. The drone should become available again.
- A drone cannot have overlapping missions (by scheduled time).
- Missions cannot be scheduled in the past.

3. Maintenance Log

Track maintenance activities performed on drones.

A maintenance log includes:

- Associated drone

- Type (ROUTINE_CHECK, BATTERY_REPLACEMENT, MOTOR_REPAIR, FIRMWARE_UPDATE, FULL_OVERHAUL)
- Technician name
- Notes (optional)
- Date performed
- Flight hours at time of maintenance

Rules:

- Creating a maintenance log should update the drone's maintenance tracking dates.
- The drone's status should reflect when it is under maintenance.
- The recorded flight hours at maintenance should be consistent with the drone's actual total flight hours (within a reasonable tolerance).

4. Dashboard (Frontend)

Build a dashboard with:

- **Fleet Overview:** Drone count by status
- **Maintenance Alerts:** Drones with maintenance due within 7 days (highlight overdue)
- **Mission View:** Upcoming and recent missions
- **Drone Detail Page:** Full drone info with mission history and maintenance history

5. Fleet Health Report

Provide an API endpoint that returns a summary of fleet health including:

- Total drone count and breakdown by status
 - List of drones with overdue maintenance
 - Number of missions in the next 24 hours
 - Average flight hours per drone
-
-

Technical Expectations

- All inputs must be validated at the API level.
 - Use proper HTTP status codes and structured error responses.
 - List endpoints must support pagination.
 - Mission list should support filtering (by status, drone, date range).
 - Use database migrations -- not auto-sync.
 - Provide a seed script with realistic test data (at least 20 drones, 50 missions, 30 maintenance logs).
-

Testing

Backend:

- Unit tests for business rule validations (state transitions, maintenance calculations, serial number format, overlap detection)
- Unit tests for the fleet health report logic
- At least one integration test covering a full mission lifecycle (create drone -> schedule mission -> complete mission -> verify state)

Frontend (E2E / Blackbox Testing):

- At least one end-to-end test covering a full user flow (e.g., create a drone, schedule a mission, transition mission through states, and verify the dashboard reflects the changes)
 - Use Cypress or Playwright
 - Tests should interact with the application as a real user would (click buttons, fill forms, assert visible outcomes) -- no internal component mocking
-

Deliverables

1. Working backend with all endpoints
 2. Working frontend with dashboard and management pages
 3. Database migrations and seed script
 4. Tests (backend and frontend)
 5. README.md with clear setup and run instructions
 6. (Bonus) Docker setup with docker-compose.yml
-

Evaluation

Your submission will be evaluated on:

Area	Weight
Code quality, structure and TypeScript usage	High
Business logic correctness	High
Testing coverage and quality	High
Frontend architecture and UX	Medium
Documentation and project setup	Medium

After submission, there will be a live session (approximately 60 min) where you will:

- Walk through your key design decisions
 - Extend the application with a small new feature
 - Debug a reported issue
 - Discuss architectural scaling questions
-

Time

This case study is designed to take approximately **8-10 hours**. We value working software with clean code over feature completeness. If you run out of time, prioritize backend business logic and tests over UI polish.

Good luck!